49

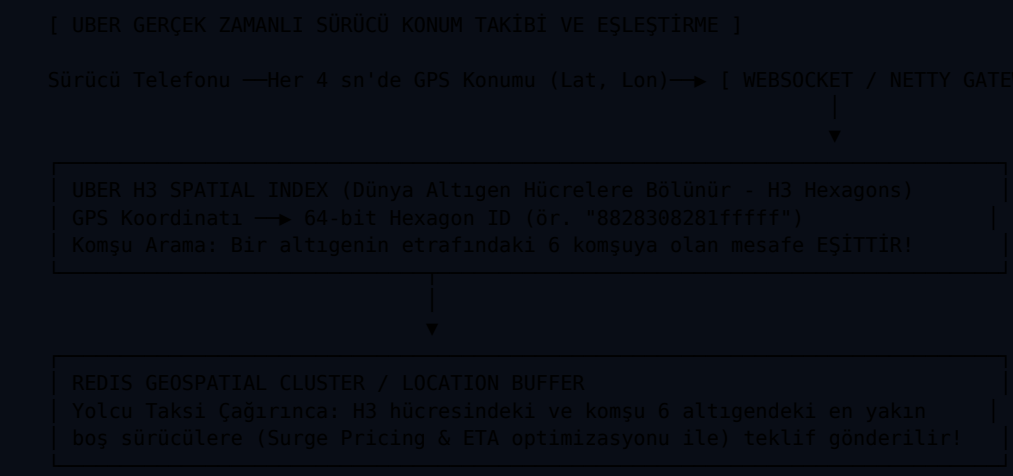
BÖLÜM: İLERI SEVIYE MAANG TASARIM PROBLEMLERI | MODULE: ADVANCED MAANG DISTRIBUTED SYSTEM DESIGNS

Gerçek Zamanlı Yolculuk Eşleştirme (Uber / Lyft): Uber H3 Altıgenleri ve Konum Takibi

Designing Ride-Hailing (Uber / Lyft): Uber H3 Hexagons & Real-Time Dispatch

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Kare bir ızgarada merkezdeki bir kutunun yan komşusu ile çapraz komşusunun mesafesi aynı değildir (çapraz komşu %41 daha uzaktır!). Bu yüzden Uber dünyayı karelere değil, arıların peteği gibi altıgenlere (H3 Hexagonal Grid) bölmüştür! Bir altıgenin etrafındaki 6 komşusunun da merkeze olan uzaklığı BİREBİR AYNIDIR. 1 milyon taksi şoförü her 4 saniyede bir 'Ben şu altıgendeyim' diye sinyal gönderir. Siz 'Taksi çağır' butonuna bastığınızda, sistem sizin bulunduğunuz altıgene ve etrafındaki 6 komşu peteğe bakar; 1 saniyede en yakın 3 taksiciyi ekranınıza düşürür!

🇬🇧 ENGLISH (ELI5)

On a square grid, the distance to an edge neighbor differs from a diagonal neighbor (diagonals are 41% further away!). To eliminate this mathematical distortion, Uber divided the entire planet into a honeycomb of hexagons (Uber H3 Spatial Index). Every neighbor of a hexagon shares the EXACT same distance to its center! 1 million drivers broadcast their GPS location every 4 seconds. When a rider taps 'Request Ride', the dispatch engine evaluates drivers in the rider's active hexagon and its 6 adjacent neighbor cells, matching the optimal driver in under 1 second!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Seviyesi Uber Mimarisi: 1) Konum Güncelleme Hacmi (Location Ingestion): 1M aktif sürücü x her 4 saniyede 1 GPS pingi = 250,000 QPS yazma yükü! Bu yükü doğrudan ilişkisel veritabanına yazmak sistemi eritir. Sürücü konumları hafif WebSocket veya gRPC bağlantılarıyla alınır ve bellek içi Redis Geospatial (`GEOADD`) veya RAM tabanlı H3 spatial index'te tutulur. 2) Neden Uber H3 (Altıgen Mimarisi)?: H3, dünyayı hiyerarşik 16 çözünürlük seviyesinde altıgenlere bölen açık kaynaklı bir kütüphanedir. Altıgenlerin tüm komşularına olan mesafeleri sabittir; bu sayede dinamik fiyatlandırma (Surge Pricing) ve sürücü arama algoritmaları çok daha homojendir. 3) Eşleştirme Motoru (Dispatch Engine): Yolcu istek attığında, H3 k-ring (örneğin 1 km yarıçapındaki altıgenler) taranır. Aday sürücüler arasından ETA (Tahmini varış süresi), sürücü puanı ve rota uygunluğuna göre en iyi eşleşme yapılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Uber Scale Spatial Architecture: 1) Ingestion Scale: 1M active drivers sending GPS pings every 4 seconds generates 250,000 writes/sec! Relational engines collapse under this load. Ingestion routes over persistent WebSockets/gRPC into distributed in-memory Redis clusters or custom H3 memory rings. 2) Why Uber H3 (Hexagonal Topology)?: Unlike square grids where diagonals are √2 times longer, hexagons exhibit uniform distance to all 6 adjacent neighbors. H3 provides 16 discrete hierarchical resolutions (Resolution 8 spans 450m, perfect for urban dispatch). Surge pricing boundaries map cleanly to H3 cells. 3) Real-Time Matchmaker: When a rider requests a pickup, the dispatcher executes an `H3.kRing(originHex, radius=2)` query. Candidate drivers are ranked via real-time routing engines (taking one-way streets, traffic congestion, and driver acceptance rates into account) before dispatching the offer.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Simüle Edilmiş Uber H3 Altıgen Eşleştirme Modeli
import java.util.*;

public class UberDispatchEngine {
    // Sürücü Modeli
    public record Driver(String driverId, String h3HexId, boolean isAvailable) {}

    private final Map<String, List<Driver>> hexagonalGrid = new HashMap<>();

    public void updateDriverLocation(Driver driver) {
        hexagonalGrid.computeIfAbsent(driver.h3HexId(), k -> new ArrayList<>()).add(driver);
    }

    // Yolcunun altıgen hücresindeki ve komşu hücrelerdeki müsait sürücüleri bul
    public List<Driver> findNearbyDrivers(String riderHexId, List<String> neighborHexIds) {
        List<Driver> candidates = new ArrayList<>();

        // 1. Yolcunun kendi hücresine bak
        for (Driver d : hexagonalGrid.getOrDefault(riderHexId, List.of())) {
            if (d.isAvailable()) candidates.add(d);
        }
        // 2. Komşu 6 altıgene bak (H3 k-Ring)
        for (String neighbor : neighborHexIds) {
            for (Driver d : hexagonalGrid.getOrDefault(neighbor, List.of())) {
                if (d.isAvailable()) candidates.add(d);
            }
        }
        return candidates;
    }
}

void main() {
    var dispatch = new UberDispatchEngine();
    dispatch.updateDriverLocation(new UberDispatchEngine.Driver("drv_1", "hex_kadikoy_c1", true));
    dispatch.updateDriverLocation(new UberDispatchEngine.Driver("drv_2", "hex_kadikoy_c2", true)); //
Komşu altıgen

    var nearby = dispatch.findNearbyDrivers("hex_kadikoy_c1", List.of("hex_kadikoy_c2",
"hex_kadikoy_c3"));
    IO.println("Eşleşen Müsait Sürücü Sayısı: " + nearby.size());
    for (var d : nearby) IO.println("• Sürücü #" + d.driverId() + " (Hücre: " + d.h3HexId() + ")");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta sürücülerin saniyelik konumlarını PostgreSQL veya MySQL'e yazacağınızı ASLA söylemeyin! Saniyede 250.000 disk yazması veritabanını felç eder. Sürücünün anlık konumu uçucu (ephemeral) veridir; sadece RAM'de (Redis) tutulur. Sadece yolculuk bittiğinde tamamlanan rota veritabanına arşivlenir.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never pitch writing raw GPS pings directly to a persistent relational database! 250,000 writes/sec will incinerate database IOPS. Current driver location is ephemeral data stored exclusively in in-memory spatial caches (Redis); only finalized trip routes commit to persistent databases.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Uber H3 altıgen ızgarası komşular arası mesafeyi eşit tutarak arama algoritmasını optimize eder.

• 250.000 QPS'lik konum akışı ilişkisel DB yerine tamamen bellek içi Redis'te tutulur.

• Yolculuk eşleştirmede H3 k-ring hücreleri taranarak en düşük ETA'ya sahip sürücü seçilir.

🎯 Key Takeaways for the Interview

• Uber's H3 hexagonal grid ensures uniform neighbor distances, eliminating diagonal distortions.

• High-frequency GPS telemetry streams into in-memory Redis clusters, bypassing disk bottlenecks.

• Dispatchers search surrounding H3 k-ring clusters to identify drivers minimizing rider ETA.